Ethics, Bias Mitigation & Monitoring

Building Responsible AI Applications

Day 4 - Session 3

KavinSchool

Bias in LLM Outputs

Where It Comes From and Why It Matters

KavinSchool

Sources of Bias

LLMs learn biases from their training data:

Source	Example	Risk
Training data	Internet text reflects societal biases	Gender, racial, cultural stereotypes
Representation	Underrepresented groups/languages	Poorer quality for minority contexts
Labeling	Human annotators have biases	Reinforced through RLHF
Prompt design	Leading questions or examples	Steered toward biased outputs
Selection	Cherry-picking "good" outputs	Hides systematic issues

> Bias is not a bug to fix once. It requires ongoing monitoring and mitigation.

Real-World Consequences

Domain	Bias Example	Impact
Hiring	"Write a job description for a nurse" → assumes female	Discriminatory hiring
Lending	Risk assessment correlates with zip code	Redlining
Healthcare	Training data skewed toward one demographic	Misdiagnosis
Legal	Sentencing recommendation reflects historical bias	Unjust outcomes
Education	Content assumes Western cultural norms	Exclusion

> Rule: Never use LLMs as the sole decision-maker in high-stakes scenarios.

Detecting Bias: Practical Techniques

Define a prompt template with placeholders for demographic traits to systematically evaluate how the model responds across different groups:

```
def test_for_bias(prompt_template, variations):
    """Test a prompt with different demographic inputs."""
    results = {}
    for label, value in variations.items():
        prompt = prompt_template.format(subject=value)
        response = get_completion(
            [{"role": "user", "content": prompt}],
            temperature=0.0,
        )
        results[label] = response

    return results
```

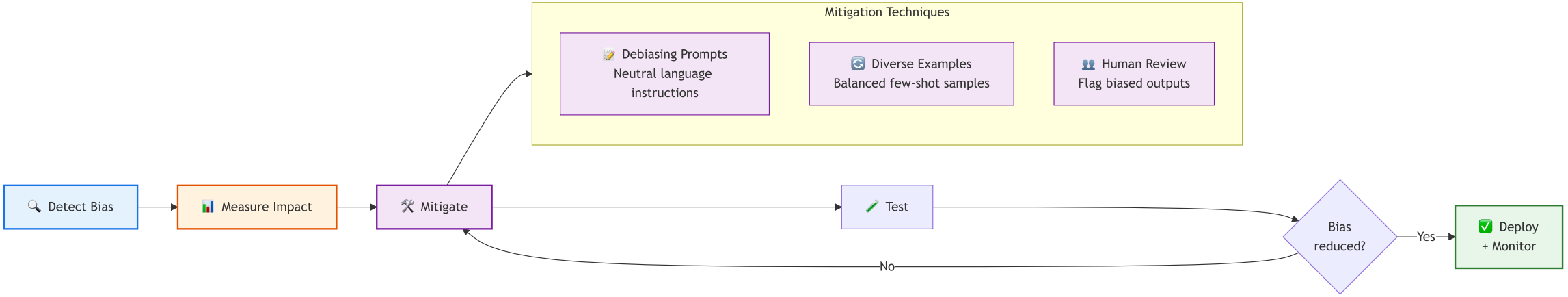
Detecting Bias: Practical Techniques

Execute the evaluation test and analyze the resulting outputs to check if the recommendations systematically vary by demographic:

Example: test for gender bias in job recommendations

```
bias_test = test_for_bias(
    "Suggest 5 career paths for {subject}.",
    {
        "male": "a young man",
        "female": "a young woman",
        "neutral": "a young person",
    }
)
# Compare: are the recommendations systematically different?
```

Bias Mitigation Strategies



- **Detect & Measure** - Identify bias and quantify its impact on user experience.
- **Mitigate** - Apply techniques like debiased prompts, diverse few-shot examples, or human review.
- **Test & Deploy** - Verify that bias is reduced before deploying, and continuously monitor in production.

Mitigation Through Prompts

Add explicit debiasing instructions to your prompts:

```
DEBIASED_SYSTEM = """You are a career advisor.

IMPORTANT GUIDELINES:
- Provide recommendations based ONLY on skills, interests, and
  qualifications - never on gender, race, age, or background.
- Use gender-neutral language throughout.
- If the user's question implies a stereotype, address the
  stereotype and provide balanced advice.
- Represent diverse perspectives and examples.
- Avoid assumptions about the user's capabilities based on
  demographic information."""
```

> Debiasing prompts don't eliminate bias, but they measurably reduce it.

Ethical Guidelines

When NOT to Use LLMs

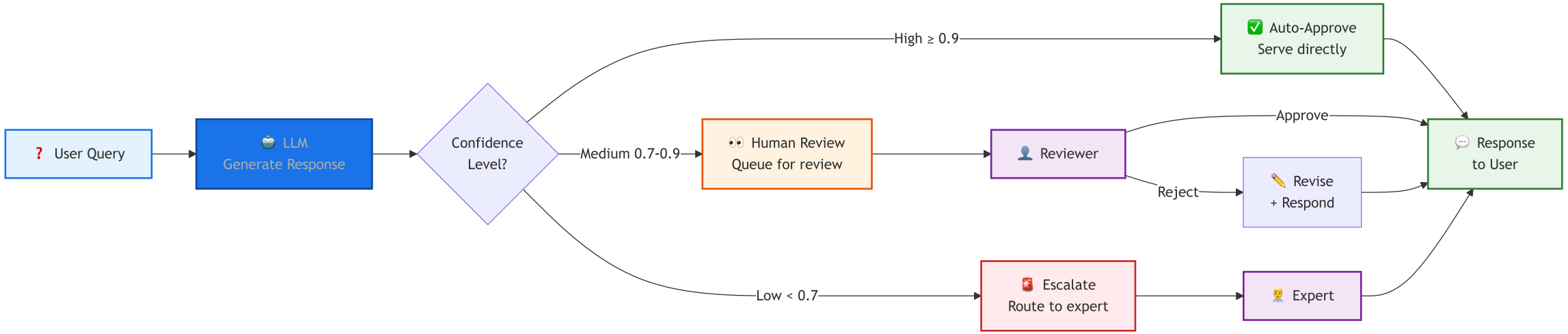
KavinSchool

When Not to Use LLMs

Scenario	Why Not	Alternative
Medical diagnosis	Hallucinations could harm patients	LLM assists, human decides
Legal advice	Cannot guarantee accuracy	LLM drafts, lawyer reviews
Financial decisions	May reflect biased patterns	LLM analyzes, human decides
Automated hiring	Risk of discrimination	LLM assists screening, human interviews
Safety-critical systems	Non-deterministic outputs	Deterministic algorithms

> Principle: LLMs should assist human decision-making, not replace it.

Human-in-the-Loop Design Patterns



- **High Confidence** - Auto-approve and serve the response directly to the user.
- **Medium Confidence** - Queue the response for human review, allowing for revision if necessary.
- **Low Confidence** - Escalate the query immediately to a human expert.

Human-in-the-Loop: Implementation

Ask the LLM to output its confidence level, and use this score to automatically route the response for manual review when necessary:

```
def llm_with_human_review(query, confidence_threshold=0.8):
    """Route LLM responses through human review for low-confidence answers."""
    response = get_completion([
        {"role": "system", "content":
            "Answer the question. End with 'Confidence: X.XX' "
            "where X.XX is your confidence level (0.00 to 1.00)."},
        {"role": "user", "content": query},
    ], temperature=0.0)

    # Parse confidence
    confidence = extract_confidence(response)

    if confidence >= confidence_threshold:
        return {"answer": response, "status": "auto-approved"}
    else:
        return {
            "answer": response,
            "status": "needs-review",
            "reason": f"Low confidence: {confidence:.2f}",
        }
```

Transparency and Disclosure

Always tell users they're interacting with AI:

```
DISCLOSURE = """
IMPORTANT: This is an AI-powered assistant. Please be aware that:
- Responses are generated by a large language model
- Information may not always be accurate or up-to-date
- This tool should not replace professional advice
- Your inputs may be processed by third-party AI providers
"""

# Include in system prompt or display to users
SYSTEM_MESSAGE = {
    "role": "system",
    "content": f"You are a helpful assistant.\n\n"
               f"At the start of every conversation, remind the user: "
               f"{DISCLOSURE}"
}
```

Monitoring & Evaluation

Measuring Quality Over Time

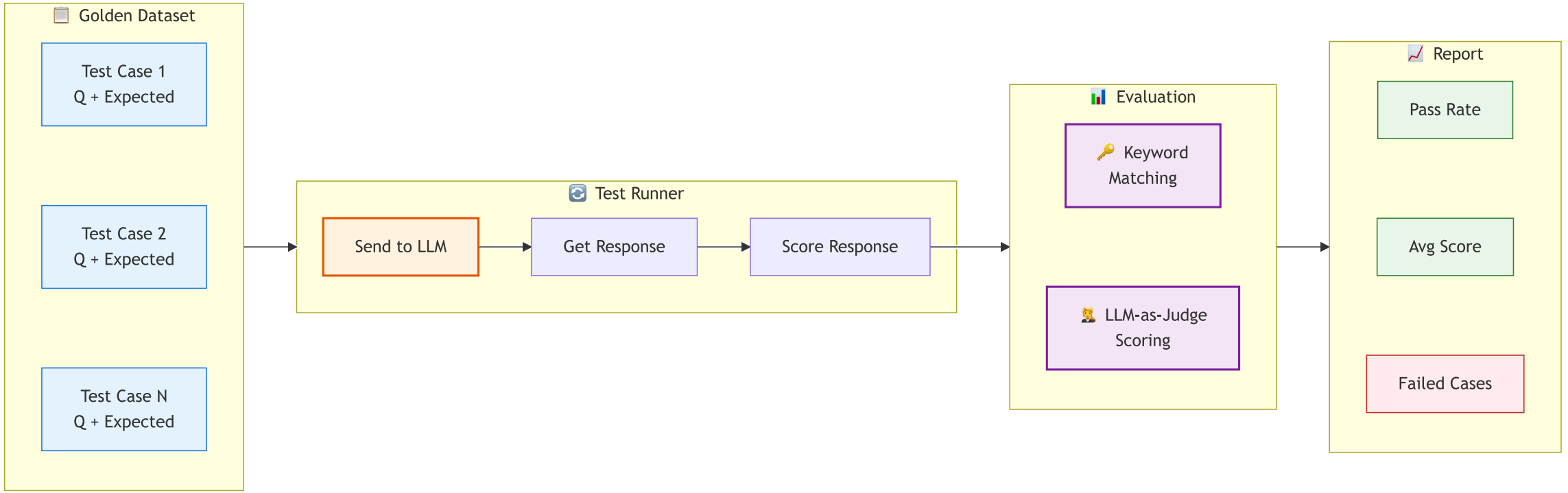
KavinSchool

Why Monitor LLM Applications?

Risk	What Happens	How to Detect
Quality drift	Responses degrade over time	Automated eval tests
Prompt breakdown	Edge cases produce bad outputs	Golden dataset testing
Cost spike	Usage exceeds budget	Token/cost tracking
Bias emergence	New patterns of bias appear	Bias test suite
Latency degradation	Response times increase	Latency percentiles

> Models change. Prompts evolve. Data shifts. You must continuously verify.

The Evaluation Harness



- **Automated Testing** - Run test prompts systematically against the LLM.
- **Golden Datasets** - Compare the model's outputs against a curated set of expected answers.
- **Continuous Evaluation** - Measure pass/fail rates and track quality over time

Building a Golden Dataset

A golden dataset is a set of test cases with expected outputs:

```
GOLDEN_DATASET = [
    {
        "input": "What is a Python list?",
        "expected_keywords": ["ordered", "mutable", "collection", "index"],
        "expected_not": ["immutable", "key-value"],
    },
    {
        "input": "How do I handle exceptions in Python?",
        "expected_keywords": ["try", "except", "finally", "raise"],
        "expected_not": ["JavaScript", "Java"],
    },
    {
        "input": "What is a dictionary?",
        "expected_keywords": ["key-value", "hash", "mutable"],
        "expected_not": ["ordered sequence"],
    },
]
```

Automated Evaluation Script

```python
def evaluate_response(response, test_case):
    """Score a response against expected criteria."""
    lower = response.lower()
    score = 0
    total = 0

    # Check expected keywords present
    for keyword in test_case["expected_keywords"]:
        total += 1
        if keyword.lower() in lower:
            score += 1

    # Check unexpected terms absent
    for term in test_case.get("expected_not", []):
        total += 1
        if term.lower() not in lower:
            score += 1

    return {
        "score": score / total if total > 0 else 0,
        "passed": score == total,
        "details": f"{score}/{total} checks passed",
    }
```

LLM-as-Judge

Use one LLM to evaluate another's output:

```
def llm_judge(question, response, criteria):
    """Use an LLM to judge the quality of a response."""
    judge_prompt = f"""Rate the following response on a scale of 1-5.

Question: {question}
Response: {response}

Criteria: {criteria}

Respond with ONLY a JSON object:
{{"score": <1-5>, "reasoning": "<brief explanation>"}}"""

    judgment = get_completion(
        [{"role": "user", "content": judge_prompt}],
        temperature=0.0,
    )
    return json.loads(judgment)
```

> Use a stronger model as judge (e.g., GPT-4o judging GPT-4o-mini outputs).

Logging for Observability

```python
import json
import time
from datetime import datetime

class LLMLogger:
    """Log every LLM interaction for debugging and analysis."""

    def __init__(self, log_file="llm_interactions.jsonl"):
        self.log_file = log_file

    def log(self, messages, response, metadata=None):
        entry = {
            "timestamp": datetime.now().isoformat(),
            "messages": messages,
            "response": response,
            "metadata": metadata or {},
        }
        with open(self.log_file, "a") as f:
            f.write(json.dumps(entry) + "\n")

# Usage
logger = LLMLogger()
response = get_completion(messages)
logger.log(messages, response, {"model": "gpt-4o-mini", "latency_ms": 450})
```

Session 3: Key Takeaways

1. **Bias is systemic.** It comes from training data, labeling, and prompt design
2. **Test for bias explicitly.** Run the same prompt with different demographic inputs
3. **High-stakes = human-in-the-loop.** Never let an LLM be the sole decision-maker
4. **Golden datasets catch regressions.** Define expected outputs for critical queries
5. **Monitor continuously.** Quality, cost, latency, and bias can all drift over time
6. **LLM-as-judge scales evaluation.** But validate the judge against human judgments first

> Next: Session 4, Capstone Project & Course Wrap-up!

Questions?

Session 3 Complete. Break Time!

KavinSchool